GRADUATION PROJECT

Project Technical Plan

AI-Based Sign Language Translator

Sign → Gloss → Natural Language Text

A Two-Stage Deep Learning Pipeline for Arabic Sign Language Translation

Team Size: 6 Engineers | Duration: 8–10 Weeks
Dataset: Ishara (Arabic Sign Language) | Language: Arabic Gloss → Arabic Text
Modules: Computer Vision · NLP · Backend API · Flutter Mobile

1. Project Overview

1.1 Problem Statement

Deaf and hard-of-hearing individuals communicate primarily through sign language, yet the vast majority of the hearing population cannot understand it. Manual human interpretation is expensive, slow, and not always available. This project builds an automated, AI-powered pipeline that bridges this communication gap by translating sign language videos into fluent natural language text in near real-time.

1.2 System Concept

The system accepts a sign language video as input and produces a natural language Arabic sentence as output through two sequential deep learning stages. The dataset used is Ishara — an Arabic Sign Language dataset consisting of 6,000 training samples with Arabic Gloss annotations and Arabic sentence translations.

Stage	Input	Model Type	Output
Stage 0 — Preprocessing	Pre-extracted frame folders (video_path)	OpenCV frame loader + MediaPipe Holistic	Keypoint sequences (.npy)
Stage 1 — Sign → Gloss	Keypoint sequences (64 × 225)	BiLSTM / Transformer Encoder	Arabic Gloss token sequence
Stage 2 — Gloss → Text	Arabic Gloss token sequence	Transformer Decoder (Seq2Seq)	Natural Arabic language sentence

1.3 Why Gloss as an Intermediate Representation?

Gloss is a simplified, word-for-word written representation of sign language. It retains the semantic content of individual signs without full grammatical structure. Using Gloss as an intermediate stage provides several critical advantages:

- Decomposition of complexity: Direct Sign → Text mapping requires a single model to simultaneously solve temporal gesture recognition and natural language generation — two fundamentally different problems. Splitting into two stages allows each model to be optimized independently.
- Data efficiency: Gloss-annotated sign language datasets (e.g., PHOENIX-2014T, CSL-Daily) are far more available than paired Sign → sentence corpora. The two-stage approach allows us to train on different datasets per stage.
- Interpretability and debuggability: Gloss output is human-readable. Errors can be diagnosed at stage boundaries, enabling targeted improvements.
- Modularity: Each stage can be retrained, swapped, or improved without affecting the other. A better Gloss → Text model can be deployed without retraining the vision module.

- Linguistic alignment: Sign languages have their own grammar (topic-comment structure, spatial references) that differs from spoken language. Gloss captures sign-level semantics, while Stage 2 handles grammatical translation into the target language.

1.4 Key Inputs & Outputs

INPUT: Frame folder path (video_path column from Ishara CSV) — pre-extracted JPEG/PNG frames
OR live camera stream captured by Flutter app
STAGE 1: Load frames from folder → MediaPipe keypoints → BiLSTM/Transformer encoder → Arabic Gloss
Example Gloss output: [هو معلم لغه اشاره]
STAGE 2: Transformer Seq2Seq decoder → Natural Arabic sentence
Example Text output: "هو مدرس لغه اشاره"
OUTPUT: Arabic natural language sentence displayed in the Flutter mobile app

2. System Architecture

2.A Computer Vision Module

2.A.1 Frame Folder Loading + MediaPipe Holistic Keypoint Extraction

The Ishara dataset provides pre-extracted video frames stored in folders referenced by the video_path column in the CSV. Each folder contains sequential JPEG/PNG frames representing one sign language sample. Frames are loaded using OpenCV (sorted by filename to preserve temporal order), then processed by MediaPipe Holistic to extract skeletal keypoints.

- Face landmarks: 468 points (facial expression, mouth shape)
- Pose landmarks: 33 points (body posture, torso orientation)
- Left hand landmarks: 21 points (finger positions and gestures)
- Right hand landmarks: 21 points (finger positions and gestures)
- Total keypoints per frame: 543 landmarks × up to 3 coordinates (x, y, z / visibility)
- Flattened feature vector per frame: 225 selected and normalized values

2.A.2 Temporal Sampling

Sign language is inherently temporal. Each sign spans multiple frames. We standardize input sequences to a fixed length of 64 frames per video clip. This is achieved by:

- Uniform sampling: Evenly sample 64 frames from the full video duration
- Padding: If the video has fewer than 64 frames, pad with zero vectors
- Truncation: If the video has more than 64 frames, apply strided sampling

Final feature shape per video: (64 frames × 225 features) = (64, 225)

2.A.3 Encoder Model — BiLSTM / Transformer

Two encoder architectures are evaluated:

Architecture	Mechanism	Strength	Weakness
BiLSTM	Bidirectional recurrent units process sequence in both directions	Captures local temporal dependencies; lighter weight	Struggles with long-range dependencies
Transformer Encoder	Self-attention across all 64 frames simultaneously	Captures global context; parallelizable	Requires more data and compute

The encoder maps the input sequence (64, 225) → a context embedding → a sequence of Gloss token indices.

2.B NLP Module

2.B.1 Vocabulary — Ishara Arabic Dataset

Vocabulary is built directly from the Ishara training CSV. The dataset contains 353 unique Arabic Gloss tokens and 695 unique Arabic sentence words across 6,000 training samples. Both vocabularies share the same special token scheme:

```
Special tokens:   PAD = 0  |  BOS = 1  |  EOS = 2  |  UNK = 3
Gloss vocabulary size:   353 unique tokens  (e.g. هو، معلم، استفهام، لغه، اشاره ...)
Sentence vocabulary size: 695 unique words  (e.g. مدرس، زميل، يعرف، اكتب ...)
Avg Gloss sequence length:  4.06 tokens  |  Max: 11 tokens  |  Min: 1 token
Avg sentence length:       3.83 words  |  Max: 8 words   |  Min: 1 word
PAD  — Padding token to fill shorter sequences to fixed max_seq_len
BOS  — Beginning-of-sequence marker prepended to every target sentence during decoding
EOS  — End-of-sequence marker; signals the decoder to stop generation
UNK  — Unknown token for words not seen during training (rare given small vocab)
```

2.B.2 Tokenization Pipeline — Arabic

- Normalize Arabic text: strip diacritics (tashkeel), normalize alef variants (أ إ آ → ا)
- Tokenize into whitespace-split word-level tokens (Arabic words are space-delimited)
- Map tokens to integer indices using the Ishara vocabulary
- Pad or truncate to max_seq_len = 12 (covers max gloss=11, max sentence=8 with BOS/EOS)
- Store as integer tensors for training — right-to-left order preserved

2.B.3 Transformer Decoder — Gloss → Text

The Transformer Decoder (Seq2Seq) architecture is used for Stage 2. It consists of:

- Encoder side: Receives Gloss token sequence as input embeddings
- Decoder side: Auto-regressively generates Text tokens
- Cross-attention: Decoder attends to Gloss encoder outputs at each generation step
- Positional encoding: Injects position information into both encoder and decoder
- Multi-head attention: 8 attention heads capture diverse linguistic relationships

2.B.4 Teacher Forcing (Training)

During training, teacher forcing is used: the ground-truth target token at position t is fed as input at position t+1, rather than the model's own prediction. This stabilizes training and accelerates convergence. During inference, the model generates auto-regressively using its own predictions.

2.B.5 Beam Search Decoding (Inference)

At inference time, greedy decoding (always picking the highest-probability token) can produce suboptimal sentences. Beam search maintains the top-k candidate sequences at each step:

- Beam width k=4 is recommended as a balance between quality and speed

- Length penalty applied to prevent preference for short sequences
- Sequence terminates when EOS token is generated or max_length is reached

2.C Backend Module

2.C.1 Framework — FastAPI

FastAPI is selected over Flask for its asynchronous request handling, automatic OpenAPI documentation generation, and type validation via Pydantic. This is critical for handling concurrent requests and streaming in real-time mode.

2.C.2 Model Loading

- Both Stage 1 and Stage 2 models are loaded into memory at server startup
- PyTorch .pt / .pth model files loaded via torch.load() with map_location
- Models set to eval() mode and wrapped with torch.no_grad() context
- GPU inference if CUDA is available; CPU fallback otherwise

2.C.3 API Endpoints

Endpoint	Method	Mode	Description
/translate/upload	POST	Offline	Accept full video file, run full pipeline, return translation
/translate/stream	WebSocket / POST	Real-Time	Accept frame batches, apply sliding window, return incremental gloss+text
/health	GET	Both	Health check — returns model load status and API version
/vocabulary	GET	Both	Returns current vocabulary size and special token mappings

2.D Flutter Mobile Application

2.D.1 Screens & Components

- Home Screen: Mode selector (Upload vs Real-Time), language selector
- Upload Mode Screen: Video file picker (gallery/files), preview player, translate button, result display
- Real-Time Mode Screen: Camera preview with live overlay, start/stop streaming button, live gloss ticker, translated sentence display
- Settings Screen: Backend URL configuration, beam width, frame rate

2.D.2 Key Flutter Packages

Package	Purpose
camera	Access device camera, capture frames at configurable FPS
video_player	Preview uploaded video files before translation
http / dio	HTTP requests to backend for upload mode
web_socket_channel	WebSocket connection for real-time streaming
flutter_riverpod	State management for async inference results
file_picker	Cross-platform file selection for video upload

2.D.3 Error Handling & Loading States

- Loading spinner shown during video upload and inference
- Error banners for network failures, model errors, or empty translations
- Retry button on failure with exponential backoff for streaming reconnects
- Empty state UI when no translation has been made yet

3. Team Distribution

The 6-member team is divided into four functional roles aligned with the four system modules. Work is structured so that CV and NLP engineers can operate in parallel once interfaces are agreed upon, with backend and Flutter integration beginning in parallel to model development.

3.1 Role Assignments & Responsibilities

Computer Vision Engineers (×2)

Responsibility	Owner
Data collection and dataset preparation (PHOENIX-2014T or custom)	CV Eng 1 + 2
MediaPipe Holistic integration and keypoint extraction pipeline	CV Eng 1
Temporal sampling logic (64-frame uniform/padded sampler)	CV Eng 1
Feature normalization and .npy file export pipeline	CV Eng 2
BiLSTM encoder model design, implementation, and training	CV Eng 2
Transformer encoder (optional upgrade) implementation	CV Eng 1 + 2
Gloss sequence output formatting and CTC / cross-entropy loss	CV Eng 2
Model evaluation (frame-level accuracy, gloss-level accuracy)	CV Eng 1 + 2
Export trained Stage 1 model to .pt file for backend integration	CV Eng 2

NLP Engineers (×2)

Responsibility	Owner
Corpus preparation: collecting Gloss–Text paired sentence data	NLP Eng 1
Vocabulary construction with PAD/BOS/EOS/UNK tokens	NLP Eng 1
Tokenization pipeline and dataset class (PyTorch Dataset)	NLP Eng 1
Transformer Seq2Seq architecture implementation (encoder + decoder)	NLP Eng 2
Teacher forcing training loop with cross-entropy loss	NLP Eng 2
Beam search decoder implementation	NLP Eng 2
BLEU score evaluation and qualitative output review	NLP Eng 1 + 2
Export trained Stage 2 model to .pt file for backend integration	NLP Eng 2

Backend Engineer (×1)

Responsibility	Owner
FastAPI project setup, routing, middleware, and CORS configuration	Backend Eng
Stage 1 + Stage 2 model loading, warm-up, and inference pipeline	Backend Eng
Offline mode endpoint: video upload, preprocessing, full pipeline	Backend Eng
Real-time mode endpoint: WebSocket frame ingestion, sliding window logic	Backend Eng
JSON response schema design and error handling	Backend Eng
Docker containerization (optional) and deployment configuration	Backend Eng
API documentation and integration support for Flutter engineer	Backend Eng

Flutter Engineer (×1)

Responsibility	Owner
Project setup, routing, and state management with Riverpod	Flutter Eng
Upload mode: file picker, video preview, HTTP upload, result display	Flutter Eng
Real-time mode: camera access, frame capture, WebSocket streaming	Flutter Eng
UI design: home, upload, real-time, and settings screens	Flutter Eng
Loading states, error handling, and retry logic	Flutter Eng
Integration testing with live backend API	Flutter Eng

3.2 Module Dependencies

CV Module → Backend: CV engineers export Stage 1 model (.pt) + preprocessing code
NLP Module → Backend: NLP engineers export Stage 2 model (.pt) + vocabulary file + beam search logic
Backend → Flutter: Backend engineer provides API docs + working endpoints
Flutter → Backend: Flutter engineer requires stable endpoint URLs and response schemas

PARALLEL WORK ENABLED:

- CV and NLP teams work in parallel from Week 1 — no cross-dependency until integration
- Flutter engineer can build UI stubs and mock API responses before backend is complete
- Backend engineer can build endpoint scaffolding with dummy models early on

4. Two Operation Modes

4.A Offline Mode (Video Upload)

In offline mode, the user selects a pre-recorded sign language video from their device and submits it for translation. This mode prioritizes accuracy over latency.

4.A.1 Flow

- User selects video file in Flutter app (.mp4 / .avi / .mov, max size: 50 MB recommended)
- Flutter app sends video to backend via HTTP POST /translate/upload (multipart/form-data)
- Backend saves video temporarily, runs OpenCV frame extraction
- MediaPipe Holistic processes each frame → extracts (N, 225) keypoint array
- Temporal sampler normalizes to (64, 225) via uniform sampling or padding
- Stage 1 model (encoder) runs inference → produces Gloss token sequence
- Stage 2 model (Transformer decoder + beam search) translates Gloss → natural language text
- Backend returns JSON: { gloss: [...], text: "...", confidence: 0.91 }
- Flutter displays Gloss sequence and final translated sentence

4.A.2 Advantages

- Full video available: optimal temporal sampling quality
- Simpler pipeline: no streaming, buffering, or window management complexity
- Suitable for accessibility tools, educational applications, documentation

4.B Real-Time Mode (Frame Streaming)

In real-time mode, the camera captures frames continuously and streams them to the backend. The system produces incremental translations as the user signs, providing near-live feedback.

4.B.1 Flow

- Flutter opens WebSocket connection to backend /translate/stream
- Camera captures frames at 15–30 FPS and sends compressed JPEG frames over WebSocket
- Backend receives frames and maintains a sliding window buffer of the last 64 frames
- Every N new frames (e.g., N=8), the backend runs Stage 1 inference on the current 64-frame window
- If Gloss output changes significantly from previous prediction, Stage 2 is triggered
- Backend streams back { gloss: [...], text: "...", partial: true } incrementally
- Flutter updates the live gloss ticker and sentence display in real time
- On WebSocket close, backend runs final full-pipeline inference on the complete buffer

4.B.2 Sliding Window Design

Window size: 64 frames (fixed — matches model input shape)
Stride / step: 8 frames (new prediction every 8 incoming frames)
Buffer type: Circular FIFO queue — oldest frames dropped as new ones arrive
Inference rate: ~2–4 Hz at 30 FPS input (every 8th frame triggers inference)
Latency target: < 500ms from sign completion to text display

4.B.3 Technical Challenges of Real-Time Inference

Challenge	Description	Mitigation Strategy
Inference Latency	Stage 1 + Stage 2 must complete within < 300ms per window	Use GPU on backend; optimize model with TorchScript or ONNX
Frame Drop Handling	Network jitter may cause missing frames in the buffer	Client-side timestamp buffering; interpolate or skip gracefully
Partial Sign Problem	Sliding window may capture a sign mid-gesture at boundaries	Overlap windows by 50%; post-process duplicate Gloss predictions
Gloss Stability	Predictions may flicker between similar Gloss sequences	Majority voting or confidence threshold gating before Stage 2
Network Bandwidth	Streaming 30 FPS JPEG frames can consume 1–5 MB/s	Downsample to 15 FPS; resize frames to 320×240 before sending
Battery & Heat	Continuous camera + WebSocket drains mobile battery	Allow user to set FPS limit; release camera on background

5. Model Training Pipeline

5.1 Ishara Dataset Structure

The Ishara dataset is an Arabic Sign Language dataset. The training CSV (train.csv) has the following structure:

Column	Type	Description	Example
sample_id	string	Unique identifier for each sample	00_0001
video_path	string	Path to folder containing pre-extracted video frames	/kaggle/input/.../00/00_0001
gloss	string	Arabic Gloss annotation (space-separated sign tokens)	هو معلم لغه اشاره
sentence	string	Natural Arabic language translation	هو مدرس لغه اشاره

Total training samples: 6,000
Unique Gloss tokens: 353
Unique sentence words: 695
Avg Gloss length: 4.06 tokens (Max: 11, Min: 1)
Avg sentence length: 3.83 words (Max: 8, Min: 1)
Folder groups (topic prefixes): 15 groups (00 through 14)
Frame format: Pre-extracted image frames (JPEG/PNG) inside each video_path folder
Language: Arabic Sign Language → Arabic Text

5.2 Data Preprocessing — Frame Folder Loading

- Read train.csv using pandas; iterate over rows to get (video_path, gloss, sentence) triplets
- For each video_path: use os.listdir() + sorted() to load all frame files in temporal order
- Load each frame with cv2.imread() and convert BGR → RGB
- Recommended split: 80% train / 10% val / 10% test (stratified by topic prefix if possible)
- Since frames are pre-extracted, no video decoding step is needed — this significantly speeds up preprocessing

5.3 Keypoint Extraction

- For each frame: extract pose (33×4), face (468×3), left hand (21×3), right hand (21×3) landmarks
- Select and flatten the most discriminative 225 values (hands + pose torso/arms)
- If landmarks are not detected (e.g., hand out of frame), fill with zeros

- Normalize keypoint coordinates: subtract wrist landmark to make hand-position invariant

5.4 Saving .npy Files

Each processed video_path folder is converted to a single .npy file and saved with matching sample_id as filename:

```
Source CSV:     train.csv  →  columns: sample_id, video_path, gloss, sentence
Structure:   keypoints/
                train/
                   00_0001.npy   # shape: (64, 225), dtype: float32
                   00_0002.npy
                   ...
                val/
                test/
            vocab_gloss.json      # 353 —, ... }  5": معلم, "4": هو{ " tokens + 4 special
            vocab_text.json       # 695 —, ... } 5": زميل, "4": مدرس{ " words + 4 special
NOTE: video_path in CSV points directly to the frame folder — no video decoding needed.
```

5.5 Vocabulary Creation — Ishara

- Read all gloss and sentence columns from train.csv
- Split each field by whitespace — Arabic words are already space-delimited
- Optionally normalize: strip diacritics, normalize alef (ا→ أ إ آ), remove tatweel (-)
- Gloss vocab: 353 unique tokens → assign indices starting at 4 (after PAD/BOS/EOS/UNK)
- Sentence vocab: 695 unique words → assign indices starting at 4
- Save as vocab_gloss.json and vocab_text.json; also save combined if sharing embedding space

5.6 PyTorch Dataset Class — IsharaDataset

A custom IsharaDataset class inherits from torch.utils.data.Dataset, reading directly from the processed .npy files and the original CSV labels:

- __init__: reads train.csv, builds sample list of (sample_id, gloss_str, sentence_str)
- __len__: returns total number of samples (6,000 for training split)
- __getitem__: loads keypoints/{split}/{sample_id}.npy → Tensor(64,225); tokenizes gloss and sentence strings using vocab dicts; pads/truncates to max_seq_len=12; returns { keypoints, gloss_ids, text_ids }

5.7 DataLoader Configuration

Parameter	Stage 1 (CV)	Stage 2 (NLP)
Batch size	32	64

Parameter	Stage 1 (CV)	Stage 2 (NLP)
Shuffle	True (train only)	True (train only)
num_workers	4	4
Pin memory	True (GPU training)	True (GPU training)
Collate fn	Custom (pad gloss sequences)	Custom (pad text sequences)

5.8 Training Loop

- Optimizer: Adam with lr=1e-4, weight_decay=1e-5
- Learning rate scheduler: ReduceLROnPlateau (patience=3, factor=0.5)
- Gradient clipping: max_norm=1.0 to prevent exploding gradients
- Epochs: 50–100 for Stage 1; 30–60 for Stage 2
- Checkpointing: save best model (by validation loss) every 5 epochs
- Mixed precision training (torch.cuda.amp) for faster GPU training

5.9 Loss Functions

Stage	Loss Function	Notes
Stage 1 (Sign → Gloss)	CTC Loss (if variable-length gloss) or Cross-Entropy	CTC handles alignment-free sequence learning; CE requires forced alignment
Stage 2 (Gloss → Text)	Cross-Entropy Loss with label smoothing (ε=0.1)	Label smoothing improves generalization and reduces overconfident predictions
Both stages	Padding mask applied	Loss computed only on non-PAD tokens; PAD positions ignored in backward pass

5.10 Evaluation Metrics

- BLEU-1 to BLEU-4 (BiLingual Evaluation Understudy): Measures n-gram overlap between generated and reference translations. BLEU-4 ≥ 20 is a reasonable baseline for sign language translation tasks.
- Gloss Accuracy: Token-level accuracy between predicted Gloss sequence and ground-truth Gloss labels. Measured on validation and test sets.
- WER (Word Error Rate): Measures substitution, deletion, and insertion errors in final text output relative to reference sentence.
- ROUGE-L: Longest common subsequence overlap metric for sentence-level evaluation.

6. Deployment Plan

6.1 Saving Trained Models

- Stage 1 model: torch.save(model.state_dict(), 'stage1_encoder.pt')
- Stage 2 model: torch.save(model.state_dict(), 'stage2_decoder.pt')
- Additionally save full model config as JSON (layer dims, heads, vocab size) for reproducible loading
- Vocabulary files: vocab_gloss.json, vocab_text.json (both derived from Ishara train.csv)
- Preprocessing config: frame_count=64, feature_dim=225, normalization params, max_seq_len=12

6.2 Loading in Backend

- At FastAPI startup (lifespan event): load both model configs, instantiate model classes, load state dicts
- Use torch.device('cuda' if torch.cuda.is_available() else 'cpu')
- Run warm-up inference on dummy input to pre-compile CUDA kernels before first real request
- Expose model status via /health endpoint for readiness checks

6.3 GPU vs CPU Inference

Scenario	Device	Expected Latency	Recommendation
Development / Demo	CPU	2–8 seconds per video	Acceptable for offline mode demos
Production (cloud)	GPU (e.g., T4)	< 500ms per video	Required for real-time mode
Mobile inference (future)	CoreML / TFLite	< 200ms on-device	Advanced optimization — post-graduation scope

6.4 Docker (Recommended)

Containerizing the backend ensures reproducible deployment across machines and simplifies cloud deployment.

```
Base image:    python:3.10-slim  (or nvidia/cuda:11.8-cudnn8-runtime for GPU)
Layers:        Install system deps → pip install requirements.txt → copy app code
Volumes:       Mount /models directory for model file hot-swap without rebuild
Ports:         Expose 8000 (FastAPI) and 8001 (WebSocket if separate)
Compose:       docker-compose.yml with app + optional nginx reverse proxy
```

```
Build command: docker build -t sign-translator:latest .
Run command:   docker run --gpus all -p 8000:8000 sign-translator:latest
```

6.5 API Structure

Endpoint	Method	Request	Response
/translate/upload	POST	multipart: video file	{ gloss: [], text: str, confidence: float, duration_ms: int }
/translate/stream	WebSocket	Binary JPEG frames	{ gloss: [], text: str, partial: bool, frame_count: int }
/health	GET	—	{ status: ok, stage1: loaded, stage2: loaded, device: cuda }
/vocabulary	GET	—	{ size: int, special_tokens: {PAD:0, BOS:1, EOS:2, UNK:3} }

7. Timeline — 10-Week Project Plan

Week	Phase	CV Team	NLP Team	Backend	Flutter
1	Setup & Research	Dataset acquisition, MediaPipe env setup, test keypoint extraction on sample videos	Dataset acquisition, vocabulary analysis, tokenizer prototype	FastAPI scaffold, endpoint stubs, dummy model loader	Project setup, navigation routing, mock UI screens
2	Preprocessing	Build full extraction pipeline, export .npy files for all training samples	Build Gloss-Text dataset class, DataLoader, vocabulary finalization	Integrate preprocessing into upload endpoint with file I/O	Upload screen with file picker and video preview
3	Model Dev I	Implement BiLSTM encoder, initial training run with CTC loss	Implement Transformer Seq2Seq architecture with teacher forcing	Model loading utility, inference function stubs	Real-time screen with camera preview and frame capture
4	Model Dev II	Hyperparameter tuning, validation accuracy tracking, checkpoint saving	Training loop, beam search implementation, BLEU evaluation	Upload endpoint fully wired with Stage 1 (dummy Stage 2)	HTTP client integration for upload mode end-to-end test
5	Model Dev III	Transformer encoder experiments (optional), Stage 1 finalization	Stage 2 finalization, BLEU-4 ≥ 20 target, export model	Full pipeline integration: Stage 1 + Stage 2 working end-to-end	WebSocket client setup for streaming mode
6	Integration I	Provide .pt model + preprocessing config to backend	Provide .pt model + vocab files to backend	Full offline mode pipeline tested with real CV and NLP models	Offline mode fully integrated and tested with backend
7	Integration II	Assist backend with real-time sliding window validation	Assist backend with streaming Gloss → Text performance tuning	Real-time streaming endpoint with sliding window and WebSocket	Real-time streaming mode integrated and tested
8	Testing & QA	Edge cases: poor lighting, fast signing, partial occlusion	Edge cases: OOV words, long sentences, grammatical robustness	Load testing, error handling, latency profiling and optimization	UI polish, loading states, error banners, settings screen

Week	Phase	CV Team	NLP Team	Backend	Flutter
9	Deployment	Final Stage 1 model export and documentation	Final Stage 2 model export and documentation	Docker containerization, cloud deployment (optional), API docs	App build, APK generation, final demo device testing
10	Presentation	Prepare CV module slides and demo materials	Prepare NLP module slides and BLEU results	Prepare architecture diagram and live API demo	Prepare demo video and app walkthrough for committee

Key milestones:

- End of Week 2: All .npy data files exported and vocabulary finalized → both CV and NLP teams begin model training
- End of Week 5: Both Stage 1 and Stage 2 models producing meaningful outputs on validation set
- End of Week 7: Full end-to-end pipeline working in both offline and real-time modes
- End of Week 9: Deployment-ready build with Docker, tested on target device
- End of Week 10: Presentation rehearsed, demo recorded as backup

8. Technical Justification

8.1 Why Keypoints Instead of Raw Frames?

Criterion	Raw Frames (Pixels)	MediaPipe Keypoints
Input dimensionality	720p frame = ~1.3M pixels × 3 channels = 4M values	(64, 225) = 14,400 values — 280× smaller
Background sensitivity	Highly sensitive — model may overfit to background, lighting, clothing	Background-invariant — only body/hand skeleton is represented
Computational cost	CNN required for spatial feature extraction — very expensive	No CNN needed — keypoints already capture spatial structure
Generalization	Requires large diverse datasets to generalize across signers, locations	Signer-agnostic — coordinates normalize relative body pose
Training data needs	Needs 100k+ labeled videos for CNN robustness	Manageable with 5k–50k samples for sequential models
Privacy	Raw video stored or transmitted — PII risk for face/identity	Keypoints contain no identifiable image data

Conclusion: Keypoints deliver a compact, robust, privacy-preserving, and computationally efficient representation that allows the model to focus entirely on gesture dynamics rather than visual appearance.

8.2 Why Sign → Gloss → Text Instead of Direct Translation?

Direct Sign → Text approaches treat the entire pipeline as a single model that must learn both gesture recognition and language generation simultaneously. This creates several fundamental problems:

- Data scarcity: Paired (video, natural language sentence) datasets are rare and often small. With Ishara's 6,000 samples, the two-stage approach allows the CV module to focus on learning 353 Gloss tokens — a much simpler classification problem than direct Arabic sentence generation. The NLP module can then be trained independently on the Gloss → sentence pairs.
- Model complexity: Forcing a single model to learn temporal gesture dynamics AND grammatical sentence generation creates an excessively complex optimization landscape with poor convergence.
- Error propagation is manageable: In the two-stage approach, we can measure exactly how much error each stage introduces. If Gloss accuracy is 85% but final BLEU is only 12, we know Stage 2 is the bottleneck.
- Linguistic grounding: Sign languages are not simply encoded spoken language. They have distinct grammatical structures (non-manual markers, spatial grammar, simultaneity). Gloss

captures sign semantics while Stage 2 handles the grammatical transformation — each stage respects its own linguistic domain.

- Transfer learning potential: A trained Stage 2 Gloss → Text model can be reused or fine-tuned across different sign languages (ASL, BSL, DGS) by only retraining Stage 1.

8.3 Why Transformer for Sequence-to-Sequence Translation?

The Transformer architecture (Vaswani et al., 2017) has become the dominant approach for sequence-to-sequence tasks due to several properties that are especially relevant here:

Property	Why It Matters for This Project
Self-attention mechanism	Captures dependencies between any two positions in the sequence regardless of distance — critical for sign language where early signs can modify the meaning of later ones
Parallel computation	Unlike RNNs that process tokens sequentially, Transformer processes all positions simultaneously — dramatically faster training on GPU
Positional encoding	Injects explicit position information, allowing the model to learn word order without sequential bias
Scalability	Performance scales with more data and larger models without architectural changes — important as more training data becomes available
Pretrained models available	Stage 2 can be initialized from pretrained CAMeL-BERT or AraBERT weights for Arabic, significantly reducing data and compute required to achieve good translation quality on the Ishara 6k corpus
Cross-attention	Decoder's cross-attention over encoder output allows the text generation to selectively focus on relevant Gloss tokens — mirrors human translation attention

Conclusion: The Transformer is not just the current state-of-the-art in NLP — its properties (global attention, parallelism, scalability, pretrained availability) are specifically well-suited to the Gloss → Text translation task in this project.

8.4 Summary of Design Decisions

Decision	Alternative Considered	Why We Chose This Approach
MediaPipe keypoints	Raw CNN features from video frames	280× smaller input, background-invariant, no CNN required
64-frame temporal window	Variable-length sequences with masking	Fixes input shape for batch processing; covers ~2–3 seconds of signing
Two-stage pipeline	End-to-end Sign → Text single model	With only 6k Ishara samples, single-stage would overfit; two stages allow separate optimization of 353-token Gloss and 695-word Arabic text tasks

Decision	Alternative Considered	Why We Chose This Approach
Transformer Seq2Seq	LSTM Seq2Seq with attention	Parallel training, global attention, access to pretrained weights
Beam search (k=4)	Greedy decoding	Higher translation quality at marginal inference cost
FastAPI backend	Flask or Django	Async support for streaming, auto-docs, better performance
Flutter mobile	React Native or native iOS/Android	Single codebase for iOS + Android, good camera + WebSocket support